



DeepSeek Harness en ELECTRONICA

> Evaluación de características del harness dsh y su posible incorporación al espacio de trabajo

Resumen ejecutivo

Objetivo. Explorar las características de DeepSeek Harness (dsh), compararlas con la arquitectura determinista de 3 capas de ELECTRONICA y decidir cuáles incorporar.

Método. Se construyó primero el skill de consulta dsh (vía `flujo_repo_a_skill.py`, perfil `referencia_codigo`) a partir del repo `deepseek-ai/deepseek-harness`; este memo se redacta consultando ese skill y la especificación `docs/AGENTE_IA/dsh-agent-skill-spec.md`.

Veredicto. No se recomienda adoptar dsh como segundo harness ni migrar el kernel a Cordis. Sí se recomienda **incorporar por analogía** dos ideas concretas: (1) trazabilidad `append-only` de sesiones y (2) un gate de validación “todo es configurable por plugin”. El detalle está en la matriz de la Sección 3.

1 Qué es DeepSeek Harness

dsh es un **agent harness** open-source de DeepSeek AI. Su filosofía central es “*Everything is a plugin. Every run is traceable.*” y está construido sobre el kernel **Cordis** (gestión de plugins, montaje/desmontaje y dependencias).

- **Harness vs. Modelo:** el modelo es el “motor de razonamiento”; el harness permite al agente interactuar con su entorno, ejecutar herramientas y persistir.
- **Kernel:** Cordis gestiona el ciclo de vida (Fiber), los servicios (`ctx.*`) y los eventos (`emit/serial/bail/parallel/waterfall`).
- **Trazabilidad:** log de sesión **append-only** con todos los eventos (`turn/*`, `step/*`, `user/message`, `assistant/message`, `tool/*`); habilita reanudar, bifurcar, buscar y reproducir sesiones.

Los 4 modos de ejecución

4 modos de ejecución:

1. **Standard:** agente completo con herramientas locales (edición, shell, búsqueda, subagentes).
2. **Code:** tools expuestas vía SDK (TipoScript) para orquestar tool-calls en un programa.
3. **Minimal:** benchmark restringido (solo `bash` y `str_replace_editor`).
4. **Creator:** autoría de presets con inspección de runtime y sandboxing de plugins.

2 Comparativa con ELECTRONICA

Aspecto	DeepSeek Harness	ELECTRONICA
Filosofía	“Todo es plugin. Cada ejecución es trazable.”	3 capas deterministas: Directivas → Orquestación → Ejecución
Kernel	Cordis (DI, ciclo de vida, eventos)	Orquestador opencode + scripts execution/*.py
Selección de modelo	Configurable por agente, multi-proveedor	Determinista vía enrutador.py (descriptor → tier)
Sesiones	Log append-only + resume/fork/search/replay	Sessions/*.md por tema + .tmp/run_state.json por flujo
Extensibilidad	Plugins npm (bundles, parches, overlays)	Scripts deterministas + directivas YAML
UI	Web UI local (:3080)	CLI-first (opencode)
Lenguaje	TypeScript/pnpm + Cordis + vendor/	Python (conda) mayoritariamente

3 Matriz de decisión

Característica dsh	Descripción	Valor	Veredicto
Trazabilidad append-only	Log de eventos inmutable por sesión, reanudable	Alto	Adaptar
Sistema de plugins / bundles	Toda capacidad es un plugin montable	Medio	Adaptar (parcial)
Modos de ejecución	standard/code/minimal/creator	Medio	Descartar (freno)
Web UI	Interfaz web local	Bajo	Descarta
Framework Cordis como kernel	Sustituir el orquestador por Cordis	Bajo	Descartar
Protocolo Typert / remote	RPC tipado Host-Cliente	Bajo	Descartar
Gestión de subagentes multi-provider	codex/claude-code/acp/dsh-sdk	Medio	Evaluar a futuro

3.1 1. Trazabilidad append-only (Alta prioridad – adaptar)

dsh garantiza que *todo lo que el modelo ve es reconstruible desde el log*. Hoy ELECTRONICA guarda el estado de cada flujo multi-paso en `.tmp/run_state.json` (un solo archivo sobrescrito) y las sesiones académicas en `Sessions/*.md`.

✓ Qué incorporar por analogía

- Registrar un **log de eventos append-only** (`.tmp/session_log.jsonl`) por flujo: step/inicio, tool/llamada, resultado, timestamps. Inmutable: nunca se edita ni elimina una entrada.
- Derivar `.tmp/run_state.json` como vista (checkpoint) de ese log.
- Permitir **reanudar** un flujo interrumpido leyendo el log (resume) y **reproducir** una ejecución (replay) para diagnósticos.

Impacto: mejora la trazabilidad (principio “every run is traceable”) sin romper el determinismo (el log solo registra, no decide). Costo bajo: un helper determinista en `execution/`.

3.2 2. Plugins / bundles (Media prioridad – adaptar parcialmente)

La filosofía “todo es plugin” de Cordis inspira: cada **capacidad** (herramienta, seam, backend) es una unidad montable con configuración y dependencias declaradas.

🔥 Qué tomar prestado

- Formalizar los **seams** de ELECTRONICA: interfaces abstractas con backends intercambiables (LLM, storage, filesystem) detrás de la capa de ejecución.
- Los `execution/*.py` ya son unidades con responsabilidad única: solo falta declarar sus dependencias/entradas explícitamente (contrato de CLI) y su “montaje” en la directiva (análogo al `bundle`).

Esto **no implica** adoptar el framework Cordis (TypeScript/npm), sino el *patrón*: directivas YAML que “componen” scripts deterministas.

3.3 3. Modos de ejecución / Web UI / framework completo (Descartar)

📌 Recuerda

- Mantener **dos harness** (opencode + dsh) duplica mantenimiento y consumo de RAM (límite estricto de 4 GB del workspace).
- La Web UI requiere ejecutar un servicio Node en :3080 de forma persistente; choca con el enfoque CLI-first y de scripts deterministas.
- Emigrar el orquestador a Cordis rompería la arquitectura de 3 capas validada y todo el ecosistema de directivas/scripts ya documentado y en producción.
- El proyecto es **developer preview** con *breaking changes* frecuentes.

3.4 4. Subagentes multi-provider (Evaluar a futuro)

La lista `dsh-subagents` (`spawn`, `fork`, `acp`, `codex`, `claude-code`, `dsh-sdk`) es análoga a nuestra delegación vía `Task tool` / subagentes de opencode. No se necesita ahora, pero sí se registra como patrón a considerar si un día el orquestador delega trabajo a agentes externos de forma sistemática.

4 Recomendaciones

1. **Corto plazo:** implementar el log append-only como helper determinista en `execution/` y documentar su uso en la directiva genérica de flujos. Ver `directives/` una vez acordado el diseño.
2. **Mediano plazo:** formalizar “seams” (contratos de interfaz) en unos pocos scripts clave (LLM client, storage), sin cambiar su implementación.

3. **No hacer ahora:** adoptar dsh/Cordis como kernel, Web UI, o segundo harness.
4. **Pendiente del usuario:** prueba hands-on de los 4 modos de dsh (`npx @deepseek-ai/dsh web`) quedó pendiente de decisión.

Decisión adoptada

Incorporar **solo por analogía** (patrones y principios), no por adopción de la infraestructura. Se conserva el determinismo como guardrail: misma entrada → misma salida, y la lógica de negocio vive en `execution/*.py`, nunca en el prompt.

Basado en el skill `dsh` (2026-09-08) y la especificación `dsh-agent-skill-spec.md`. Fase B de la exploración de DeepSeek Harness.